PROJECT DOCUMENTATION

Suez Pulse

Cloud-Based Real-Time Maritime Vessel Tracking and Analytics Platform

Project design and implementation plan

DEPI Data Engineering Track

Suez Pulse connects live AIS ingestion, streaming transformation, cloud storage and interactive vessel analytics in one measurable Data Engineering workflow.

This document presents the project scope, architecture, delivery plan, responsibilities, requirements, evaluation criteria, risks and related work. It defines the implementation and evaluation framework for Suez Pulse.

Repository:

AhmedRagab010/SuezPulse

Team Members

The team consists of six Data Engineers, with responsibilities organized by workstream as shown below.

Name	Role	Workstream responsibility
Ahmed Ragab Abdallah	Data Engineer	Python ingestion, source connection and reconnect handling
Osama Abdelrahman	Data Engineer	Kafka configuration, producer reliability and integration
Doaa Mohamed	Data Engineer	PySpark ETL, schema validation and deduplication
Manar Abdelbaky	Data Engineer	PostgreSQL schema, queries and traffic aggregates
Sama Saied	Data Engineer	Streamlit dashboard, user scenarios and documentation
Abdellah Essam	Data Engineer	Azure deployment, monitoring and performance tests

Shared responsibilities

Each owner produces code, configuration, tests and a short runbook for their workstream. A second member reviews each change before integration. Ahmed and Osama review ingestion together; Doaa and Manar review data contracts; Sama and Abdellah review dashboard deployment and acceptance evidence.

Working method

Use one shared backlog with To Do, In Progress, Review and Done states. Hold two short coordination meetings each week and one weekly integration demo. Record scope changes and technical decisions in the repository. A deliverable is done when it runs from documented instructions, passes its acceptance checks and has a peer review.

Document navigation

Project Overview • Architecture Diagram • Data Flow Diagram • Technology Stack • Project Plan • Requirements • KPIs • Risk Assessment • Literature Review • References

Project Overview

Project proposal

Suez Pulse is a cloud-based Data Engineering platform designed to ingest, process, store and analyze Automatic Identification System (AIS) data from maritime vessels. AIS messages provide observations such as vessel identifiers, coordinates, speed and course. The platform will turn received observations into queryable history, latest known positions and transparent traffic indicators.

Problem statement

A live AIS stream is not directly suitable for an analytical dashboard. Messages can be repeated, incomplete, delayed or received out of order. A useful platform must preserve provenance, validate records, organize history and expose clear freshness information before drawing conclusions from vessel activity.

Objectives

- Build a complete AISStream.io → Python → Kafka → PySpark → storage → Streamlit pipeline.
- Evaluate a workload exceeding one million input records, with original observations and replayed records counted separately.
- Provide a vessel map, MMSI search, historical tracks and traffic summaries for an agreed geographic area.
- Measure data quality, processing latency, throughput and recovery behavior using reproducible tests.
- Deploy the demonstration on selected Azure resources and document setup, operation and limitations.

Scope and boundaries

The minimum viable product covers a configurable study area, supported position reports, optional static vessel enrichment, retained historical observations and read-only dashboard access. It includes validation, deduplication, monitoring, replay of captured data and basic congestion proxies.

Navigation safety, collision avoidance, guaranteed worldwide coverage, commercial service-level commitments, predictive arrival times and machine-learning models are outside the initial scope. The title uses real-time to describe streaming ingestion; the user experience is near real-time and depends on source coverage and processing delay.

Architecture Diagram

The architecture connects streaming ingestion, processing, storage and visualization. PySpark writes archive and analytical outputs to separate storage destinations; the dashboard queries PostgreSQL.

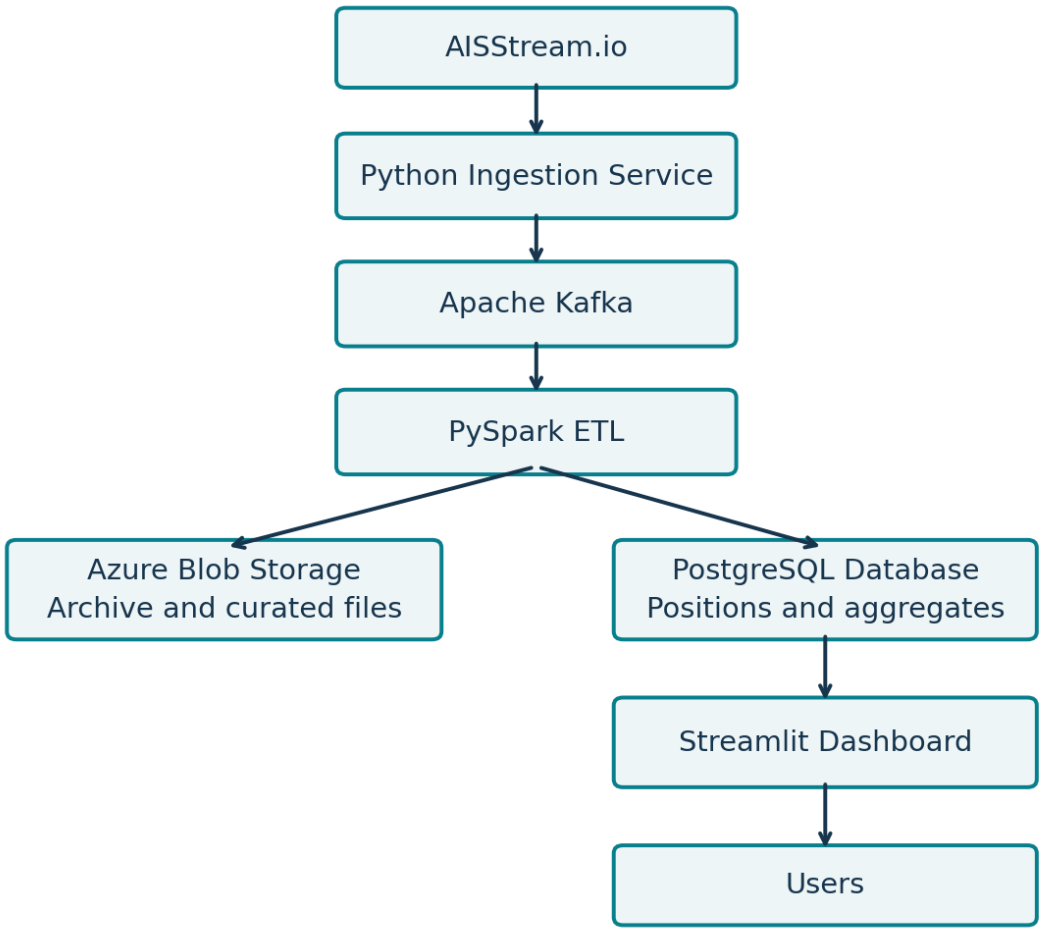


Figure 1. System architecture. Arrows show the primary direction of data movement.

Deployment and reliability boundary

Containerized ingestion, Kafka, Spark and Streamlit are designed to run on an Azure VM, with Blob Storage and PostgreSQL hosted on Azure resources. Resource sizing is based on workload profiling. A single VM or broker supports the demonstration environment but does not provide high availability.

Data Flow Diagram

This logical view shows what happens to the records. D1, D2 and D3 are data stores; numbered labels identify processing steps. User filters enter step 4 and determine read-only database queries.

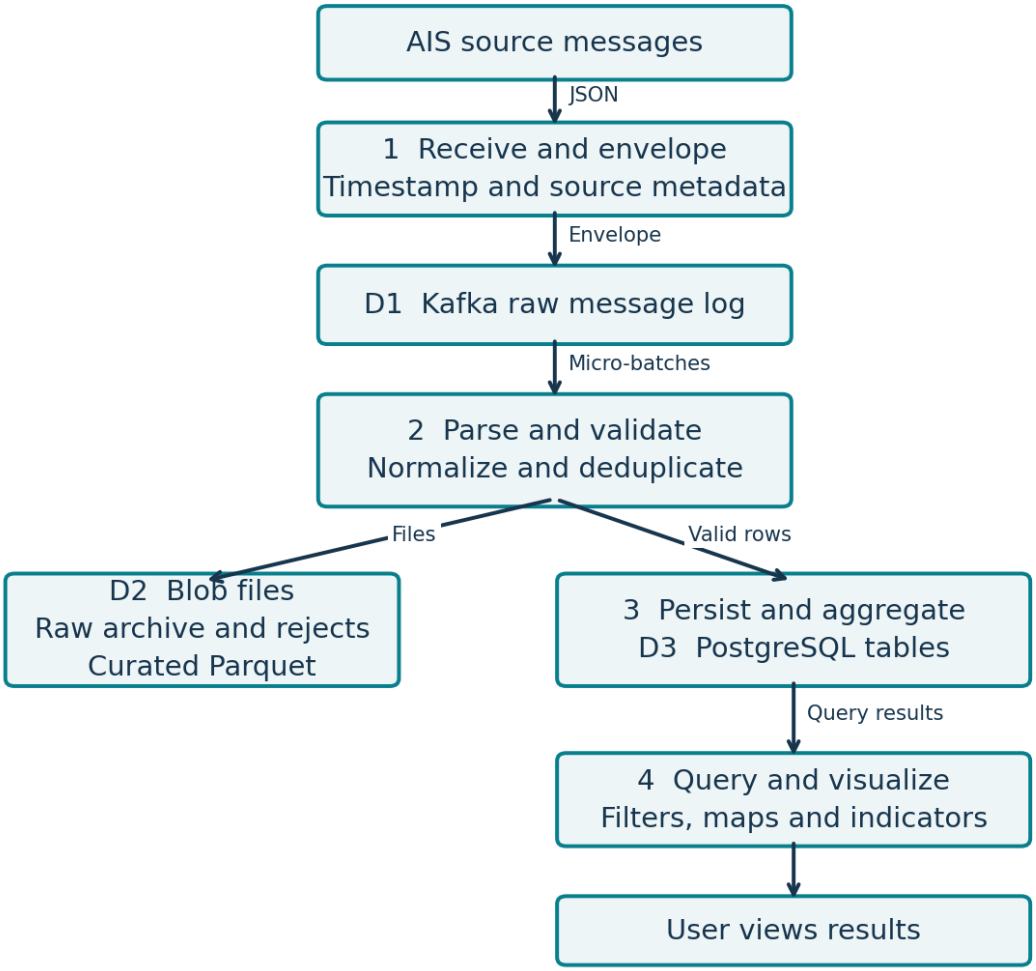


Figure 2. Logical flow from received source records to user-facing analytics.

PySpark preserves the raw envelope before validation, writes rejected records with reason codes, and stores valid rows in curated files and PostgreSQL. Dashboard filters travel back to its query layer; no user operation modifies the source data.

Technology Stack

Technology	Purpose	Implementation choice
Python	Source ingestion	Server-side WebSocket client; decode JSON; envelope and Kafka publishing.
Apache Kafka [6]	Message buffering	MMSI partition key; retained raw topic; lag and producer error monitoring.
PySpark [7]	Streaming ETL	Structured Streaming micro-batches; explicit schema, checkpoints and batch identifiers.
Azure Blob Storage [8]	Object storage	Raw envelopes, rejected records, curated Parquet and separate checkpoint paths.
PostgreSQL [9]	Analytical serving	Position history, latest positions, quality metrics and window aggregates.
Streamlit [10]	User interface	Read-only maps, filters and summary charts; bounded queries and short cache TTL.
Docker and Git	Reproducible delivery	Versioned configuration, reviewed changes and pinned dependency versions.

Data model

vessel_positions stores record_id, MMSI, source_time when available, received_at, processed_at, latitude, longitude, speed_over_ground, course_over_ground and quality flags. vessel_latest stores the newest valid observation per MMSI; older arrivals must not overwrite newer positions. traffic_windows stores counts by area and time window. pipeline_metrics stores ingest counts, rejects, duplicates, lag and sink outcomes.

Use UTC timestamps and document source-time semantics. Do not treat the AIS seconds-within-minute field as a complete date and time. Store unavailable measurements as null with a quality flag rather than as valid zero values. Add indexes on MMSI and observation time; evaluate time partitioning from query plans. PostGIS is optional if polygon queries become a requirement.

Persistence and replay

Use a stable observation fingerprint based on canonical source fields, excluding local receipt time. Keep Kafka topic, partition and offset for lineage. Use transactional upserts and deterministic file or batch identifiers so retries do not duplicate analytical results. Separate sinks are not one atomic transaction: record completion per sink and reconcile partial batches. Checkpoints alone do not guarantee exactly-once delivery across both sinks.

Project Plan

The implementation plan spans eight weeks from project kickoff. Parallel bars indicate overlapping work, not independent completion.

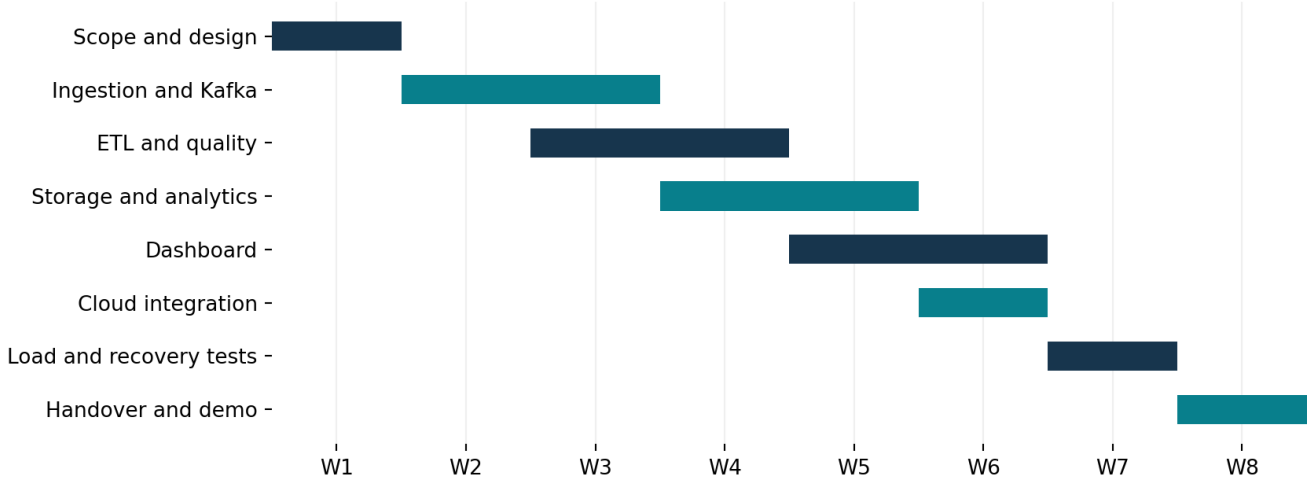


Figure 3. Relative Gantt chart for the implementation baseline.

Milestone	Due	Deliverable and exit criterion
M1 Scope baseline	W1	Defined study area, requirements, schema and repository structure.
M2 Live ingestion	W3	Python → Kafka live flow; disconnect and reconnect demonstrated.
M3 Trusted storage	W5	Validation, raw archive, PostgreSQL and replay reconciliation working.
M4 Integrated dashboard	W6	Cloud demo with map, filters, history and documented traffic metrics.
M5 Evaluation	W7	More than 1M records evaluated; latency, quality and recovery report.
M6 Handover	W8	Reviewed documentation, runbook, test evidence and final presentation.

Dependencies and change control

A sample source message and schema are required before ETL integration. Stable serving tables are required before dashboard acceptance. Cloud access is required in Week 1. If delivery time becomes limited, the core pipeline takes priority over optional static enrichment and PostGIS. Changes are recorded with their owner, reason, requirement impact and revised milestone.

Infrastructure Allocation

Use local containers during development and one shared cloud demonstration environment for integration. Start with one Kafka broker and one Spark execution environment; benchmark before increasing resources. Provision one Blob account with separate logical paths for raw, curated, rejected and checkpoint data, plus one PostgreSQL database.

A provisional retention policy is 24 hours for Kafka and 30 days for the demonstration archive and database, subject to capacity and permitted use. Size storage using measured bytes per record × expected records per day × retention, plus indexes, replication where configured and free-space headroom.

Delivery evidence

Keep the setup guide, configuration example without secrets, schema migrations, source sample, test fixtures, KPI results, risk log and demonstration script in the repository. The final report must state hardware, software versions, input volume, test duration and known limitations so another reviewer can reproduce the evaluation.

Requirements

Stakeholder analysis

Stakeholder	Need	Validation method
Maritime analyst or researcher	Find vessels and compare traffic within an area.	Scenario walkthrough with selected reviewers.
Port or fleet observer	See recent positions and understandable traffic indicators.	Review freshness and metric definitions; prospective user only.
DEPI mentor or evaluator	Assess architecture, reproducibility and engineering outcomes.	Milestone reviews and final evidence pack.
Development team	Clear ownership, stable interfaces and recoverable operation.	Integration demos and recovery exercise.
Data provider and cloud account owner	Permitted data usage and protected credentials.	Access and data-use review.

User stories

- US1 — As an analyst, I want to search by MMSI and filter by area so I can locate relevant vessels.
- US2 — As a researcher, I want to select a vessel and time range so I can inspect its historical route.
- US3 — As an observer, I want traffic counts and a clearly defined congestion proxy so I can compare activity over time.
- US4 — As an operator, I want source status, lag and reject counts so I can identify pipeline problems.
- US5 — As an evaluator, I want documented replay and benchmark steps so I can reproduce project results.

Requirements gathering approach

Requirements are derived from the project scope and the user scenarios defined in this document. Week 1 includes reviewing user stories with the team and mentor, inspecting sample messages and defining the study area. Each requirement has an ID, priority, acceptance criterion and responsible workstream. Requirements are reviewed after the first live-data demonstration.

Use Cases and Functional Requirements

Core interaction scenarios

UC1 Find a vessel — With the dashboard available, the analyst enters an MMSI or selects an area. The system validates inputs, queries latest positions and shows matching markers with observation age. If no record exists, it displays a clear no-data message; a stale position remains labeled as stale.

UC2 Inspect history — The analyst selects a vessel and bounded UTC time range. The system returns ordered observations and plots the track with gaps visible. Invalid ranges are rejected; empty results are explained. Large requests are limited or sampled with the sampling rule disclosed.

UC3 Review traffic — The observer selects an area and window. The dashboard shows distinct vessel counts, low-speed share and data freshness. Sparse or missing data are identified and must not be presented as zero traffic.

UC4 Recover processing — After a processing failure, the operator restarts from checkpoints, verifies replay and reconciles both sinks. If Kafka retention has expired, captured Blob data may be reprocessed. Source observations never received cannot be recovered by this workflow.

ID	Requirement and acceptance criterion	Priority
FR01	Subscribe to a configurable AISStream area server-side; receive supported position messages and report connection state.	Must
FR02	Publish source envelopes to Kafka with receipt time and lineage; log producer failures and acknowledgments.	Must
FR03	Validate schema, MMSI and coordinate ranges; normalize UTC and units; retain reject reasons and deduplicate retries.	Must
FR04	Persist raw, rejected and curated files to Blob and valid history to PostgreSQL; reconcile a replay without duplicate rows.	Must
FR05	Serve latest positions, MMSI search and bounded history through Streamlit; show freshness and no-data states.	Must
FR06	Compute documented area-window vessel counts and congestion proxies; verify them against a hand-checked fixture.	Must
FR07	Expose ingest, reject, duplicate, lag and sink metrics; supply reproducible tests for more than 1M input records.	Must
FR08	Enrich vessel names or types from supported static reports when available; show missing metadata explicitly.	Should

Nonfunctional Requirements

ID	Category	Acceptance criterion
NFR01	Performance	Meet K2 and K3 latency targets under the documented benchmark profile; record p50 and p95.
NFR02	Scalability	Evaluate more than 1M inputs and measure throughput, memory and backlog; do not infer scale from volume alone.
NFR03	Reliability	Recover from a controlled processor restart within 5 minutes; reconcile acknowledged Kafka inputs after draining.
NFR04	Security	Keep keys out of code, logs and UI; use server-side secrets, private data services, encrypted connections and least-privilege database roles.
NFR05	Usability	At least 4 of 5 representative testers complete vessel search and history tasks without assistance.
NFR06	Maintainability	A reviewer can start the pinned environment, apply migrations and run smoke tests from the runbook.
NFR07	Observability	Record source state, processing lag, write failures and quality counts; distinguish outages from genuine low traffic.
NFR08	Data integrity	Prevent older observations overwriting latest state; apply unique keys and independently reconcile both outputs.

Traffic metrics and interpretation

Active vessel count is the number of distinct MMSIs with a valid observation inside the selected area during a 15-minute UTC window. The proposed low-speed share is the number of these vessels whose latest observation in that window has speed below 1 knot, divided by vessels with known speed. Display both numerator and denominator.

This low-speed share is a congestion proxy, not verified port waiting time. Anchored, berthed and naturally slow vessels can increase it. Results include the area, time window, coverage limitations and sample size. Windows with fewer than 10 speed-known vessels are flagged as low evidence. The thresholds are analytical design assumptions, not validated maritime standards.

Acceptance traceability

FR01–FR02 support US1 and US4 and are checked at M2. FR03–FR04 and NFR03/NFR08 are checked at M3 using invalid-message and retry fixtures. FR05–FR06 support US1–US3 and are checked at M4. FR07 and performance requirements support US5 and are evaluated at M5. NFR04 and NFR06 are release gates at M6.

KPIs

The values below define performance and acceptance targets; they are not reported test results. p95 means that 95% of measured observations fall at or below the stated value. The evaluation protocol defines how each target will be assessed.

ID and metric	Target	Measurement and owner
K1 Evaluation volume	>1,000,000 input records	Count inputs, valid unique records, duplicates and rejects separately. Identify replayed versus original observations. Doaa.
K2 Pipeline latency	p95 ≤15 seconds	received_at to PostgreSQL commit acknowledgment, recorded by batch. Source transmission delay is excluded. Doaa / Manar.
K3 Dashboard response	p95 ≤3 seconds	Filter action to rendered result for 100 interactions across 5 sessions. Report cache conditions. Sama.
K4 Sustained processing	≥100 records/second	Replay 1.1M inputs at 100 records/s; no growing backlog over the final 30 minutes. Abdellah / Osama.
K5 Record accountability	100% reconciled	After drain: acknowledged inputs = unique accepted + duplicates + quarantined; unexplained loss = 0. Doaa.
K6 Curated validity	100% rule compliance	Check required fields and domain constraints on persisted analytical rows; raw rejection rate reported separately. Manar.
K7 Demo availability	≥99% over 8 hours	Successful one-minute app health probes / all probes. Report upstream freshness separately. Abdellah.
K8 Recovery	≤5 minutes	Time from controlled Spark termination to resumed processing; verify final sink counts. Osama / Abdellah.
K9 User task success	≥80%	At least 4 of 5 testers independently complete search and track inspection. This is usability, not market adoption. Sama.

Benchmark protocol

Use a documented dataset and fixed configuration, with a 10-minute warm-up before latency sampling. Run the 1.1M-record replay for about 3.1 hours at the proposed input rate, then drain all pending batches. Run three trials and report each trial plus the median result. Record CPU, RAM, software versions, payload sizes, storage size, cache state and retry counts. Use separate controlled invalid and duplicate fixtures; do not mislabel replay as new live observations.

Risk Assessment

Likelihood and impact are qualitative planning judgments: L = low, M = medium, H = high. Review the register weekly and update it with observed evidence.

Risk	L / I	Mitigation and contingency	Owner
Source outage or missing coverage	H / H	Reconnect with backoff; monitor last receipt. Use labeled captured replay for demos. Disclose unrecoverable source gaps.	Ahmed
Slow consumption and source drops	M / H	Bound queues, monitor producer errors and lag, and narrow the study area. No complete worldwide-capture claim.	Osama
Invalid or out-of-order AIS data	H / H	Schema checks, null handling, quarantine, stable keys and latest-by-time upserts. Track late data explicitly.	Doaa
Partial writes across both sinks	M / H	Track batch outcomes, retry idempotently and reconcile. Reprocess captured files if the Kafka window expires.	Doaa / Manar
Database or dashboard bottleneck	M / M	Batch writes, indexes, aggregates, query limits and bounded caching; profile before resizing.	Manar / Sama
Credential exposure or public services	M / H	Exclude secrets from Git; restrict ports and roles; rotate leaked credentials and inspect access logs.	Abdellah
Misleading congestion interpretation	H / M	Publish formula, sample size and assumptions; label as a proxy and seek domain review.	Sama
Integration delay or member absence	M / H	Shared contracts, paired reviews, early integration and 48-hour team contingency. Reassign open tasks.	Whole team
Data-use or reuse restrictions	M / H	Review provider terms and repository licenses before retention, redistribution or copying code. Limit demo exports.	Ahmed / Sama

Escalation and release readiness

Escalate loss of cloud access, suspected secret exposure or an unreconciled sink mismatch immediately. The final release requires a working core flow, passed security and integrity gates, recorded KPI results and disclosed open risks. A failed target remains a reported limitation with a corrective action; it is not silently removed from the evaluation.

Literature Review

Review of existing solutions

Four related AIS projects were reviewed through their public README documentation on 19 October 2026. The review evaluates documented designs rather than independently verified execution or performance. Strengths and trade-offs distinguish documented features from architectural interpretation.

DeepDarshak Data Pipeline

DeepDarshak documents a CSV-based AIS pipeline with PostgreSQL/PostGIS, Dagster, dbt and FastAPI. It includes transformations, rule-based anomalies, map previews and observability [1]. Its strength is the separation of ingestion, orchestration, transformation and serving. For Suez Pulse, the CSV-oriented path would need adaptation to continuous WebSocket ingestion. The reviewed description does not establish a comparable million-record performance result. We will reuse the separation-of-concerns principle and explicit quality rules, rather than assume identical deployment needs.

AIS Tracker

AIS Tracker describes a Go service and PostgreSQL-based platform ingesting live AISStream traffic. It stores raw data, position history and last-known positions, with spatial analysis, a map and API [2]. Its compact service design and database-centric analytics are useful references. The README also states that a bounded ingest channel can drop messages when full: a relevant reliability trade-off. Suez Pulse will evaluate explicit buffering, loss visibility and replay, while recognizing that a Kafka/Spark stack adds operational overhead.

AIS Demo App

AIS Demo App uses FastAPI, React and PostgreSQL/PostGIS to explore imported AIS files. It documents latest positions, date filters, vessel search, historical tracks and density bins [3]. It is a useful example of translating stored observations into user-facing features. Its documented ingestion workflow is file-based, so a live streaming producer is an adaptation needed for our scope. Suez Pulse will apply bounded historical queries and visible filtering while using Streamlit for its demonstration interface.

Smart Maritime Vessel Traffic and Port Intelligence Platform

This repository describes Kafka, Spark Structured Streaming, HDFS, PostgreSQL/PostGIS, Airflow and Superset, with live and historical ingestion and additional analytics [4]. It is the closest architectural reference for Suez Pulse. The breadth of its service stack introduces more integration and operational work, an architectural trade-off rather than a measured defect. Suez Pulse proposes a narrower Azure Blob and Streamlit implementation, prioritizing documented quality and reproducible evaluation before advanced analytics.

Comparative Analysis

Solution	Documented approach	Lesson or trade-off
DeepDarshak [1]	CSV ingestion; orchestrated SQL and geospatial transformations.	Clear separation of responsibilities; live ingestion requires adaptation.
AIS Tracker [2]	Live feed; database-centered storage and analytics; Go service.	Compact deployment; bounded input queues can lose messages.
AIS Demo App [3]	Imported files; API, historical tracks and map views.	Useful interaction patterns; live stream integration is outside the documented import flow.
Smart Maritime [4]	Kafka/Spark plus orchestration, spatial storage and BI.	Strong streaming reference; broader stack increases integration scope.
Suez Pulse	Python/Kafka/PySpark; Azure Blob, PostgreSQL and Streamlit.	Narrow educational scope with explicit quality, latency and replay evaluation.

Project gap and proposed contribution

The reviewed projects already demonstrate many elements of AIS ingestion, structured storage and visualization. Suez Pulse therefore does not claim that these capabilities are novel or absent from other solutions. Its proposed contribution is a clearly scoped educational implementation connecting live ingestion to validated records, cloud storage and an accessible analytical interface.

The distinguishing emphasis is a traceable evaluation: more than one million input records, separate accounting for duplicates and rejects, measured pipeline latency, controlled recovery tests and congestion indicators with explicit definitions. These are planned deliverables, not evidence of superiority over the reviewed projects.

How the review shapes implementation

Adopt modular responsibilities from DeepDarshak, a latest-position serving table from AIS Tracker, bounded history and filtering patterns from AIS Demo, and streaming pipeline concepts from Smart Maritime. Keep the baseline technology choices consistent with available resources. Assess licenses before code reuse and cite any reused implementation directly.

Evidence limitations

The documentation review does not provide a common benchmark, and the repositories were not deployed for this comparison. Comparative performance claims would require pinned commit IDs, configuration and license inspection, and execution of the same workload under comparable resources. The contribution stated here is therefore limited to the documented project scope and evaluation approach.

References

References [1]–[4] document the related projects. References [5]–[10] support service constraints and the technical design. All ten links were checked on 19 October 2026. Online documentation may change; implementation dependencies should be pinned to specific versions.

[1] tommoirangcha. DeepDarshak Data Pipeline. GitHub repository README.

https://github.com/tommoirangcha/deepdarshak-data-pipeline

[2] thanderoy. AIS Tracker. GitHub repository README.

https://github.com/thanderoy/ais-tracker

[3] HoangSonDeveloper. AIS Demo App. GitHub repository README.

https://github.com/HoangSonDeveloper/ais-demo

[4] M-H26. Smart Maritime Vessel Traffic and Port Intelligence Platform. GitHub repository README.

https://github.com/M-H26/SMART-MARITIME-VESSEL-TRAFFIC---PORT-INTELLIGENCE-PLATFORM

[5] Apache Software Foundation. Apache Kafka 4.3 documentation.

https://kafka.apache.org/43/

[6] Apache Software Foundation. Structured Streaming Programming Guide.

https://spark.apache.org/docs/latest/streaming/apis-on-dataframes-and-datasets.html

[7] Microsoft Learn. Introduction to Azure Blob Storage.

https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blobs-introduction

[8] PostgreSQL Global Development Group. PostgreSQL documentation, Chapter 11, Indexes.

https://www.postgresql.org/docs/current/indexes.html

Source dependency note

AISStream supplies a live server-side WebSocket feed with geographic filtering. Its documentation states that delivery is not durably replayed and that no uptime SLA is provided [5]. Suez Pulse must therefore distinguish source availability from internal pipeline availability and preserve the observations it actually receives.